

EC 441 Linux VM Setup Guide

EC 441 Spring 2026 — Networking Tools Environment

Created: 2026-04-13

This guide sets up a Ubuntu 24.04 Linux virtual machine on your laptop using **Multipass**, a lightweight VM manager made by Canonical (the company behind Ubuntu). The VM gives you a full Linux environment with all the networking tools used in L21–L22 (and available for your own use throughout the course).

Why Linux specifically? Professional network engineers diagnose and monitor networks almost exclusively using Linux and BSD tools. Enterprise routing infrastructure runs Linux or BSD under the hood — Juniper JunOS (a major backbone router platform) is built on FreeBSD; Arista EOS runs Linux; firewall appliances like pfSense and OPNsense are FreeBSD-based. The tools you will learn here — `tcpdump`, `ping`, `traceroute`, `ss`, `ip`, Wireshark, Scapy — are the same tools a network engineer uses on a router’s shell or in a network operations center. This is not a simulation of professional practice; it is professional practice.

Why a VM and not your laptop’s OS directly? Several tools — Scapy (raw packet crafting) and Mininet (network emulation) — require Linux kernel features (raw sockets, network namespaces) that are not available on macOS or Windows, even through WSL2. A VM gives you a real Linux kernel on any host OS.

Step 1 — Install Multipass

Multipass runs on macOS, Windows, and Linux. Install it for your platform:

macOS (Apple Silicon or Intel)

Requires [Homebrew](#). If you don’t have it:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install Multipass:

```
brew install multipass
```

Verify:

```
multipass version
```

Windows 10 / 11

Download the installer from [multipass.run](#) and run it.

- **Windows Pro / Enterprise:** Multipass uses Hyper-V automatically. You may need to enable Hyper-V in Windows Features if it is not already on.
- **Windows Home:** Multipass uses VirtualBox as a fallback. VirtualBox will be installed automatically if not present, or you can install it first from [virtualbox.org](#).

After installation, open **PowerShell** or **Windows Terminal** and verify:

```
multipass version
```

Linux

```
sudo snap install multipass
```

Verify:

```
multipass version
```

Note for Linux users: you may be able to run the provisioning script (Step 3) directly on your host system instead of inside a VM. The VM is still recommended to keep the tools isolated and avoid version conflicts.

Step 2 — Create and Provision the VM

Download `ec441_setup.yaml` from the course website and run the following command in your terminal (macOS/Linux) or PowerShell (Windows). This creates the VM **and** installs all tools automatically — no further steps needed inside the VM.

```
multipass launch --name ec441 --cpus 2 --memory 4G --disk 20G \
  --cloud-init ec441_setup.yaml 24.04
```

This downloads the Ubuntu 24.04 image (~600 MB, one time only), boots the VM, and runs the full provisioning script unattended. **It takes 10–15 minutes.** Your terminal will return to a prompt when it is done.

Verify the VM is running:

```
multipass list
```

You should see `ec441` with state `Running`.

Step 3 — Open a Shell in the VM

```
multipass shell ec441
```

You are now inside the Linux VM. Your prompt will look like:

```
(ec441) ubuntu@ec441:~$
```

The `(ec441)` prefix confirms the Python virtual environment is active. You are ready to use all the course tools.

What the Setup Script Does (Reference)

The `ec441_setup.yaml` file provisions the VM automatically. For reference, here is what it installs and why. You do not need to run any of this manually.

```
#!/bin/bash
set -e

echo "==> Updating package lists"
sudo apt-get update -qq

echo "==> Pre-answering interactive installer prompts"
# Wireshark: allow non-root capture (answer: Yes)
echo "wireshark-common wireshark-common/install-setuid boolean true" | sudo debconf-set-selections
# iperf3: do not install as a system daemon (answer: No)
echo "iperf3 iperf3/start_daemon boolean false" | sudo debconf-set-selections

echo "==> Installing system packages"
sudo apt-get install -y -q \
    # Core networking CLI tools
    iproute2 \
    iputils-ping \
    traceroute \
    dnsutils \
    net-tools \
    # Capture and analysis
    tcpdump \
    tshark \
    wireshark-common \
    # Throughput and scanning
    iperf3 \
    nmap \
    # Raw connections and crypto
    netcat-openbsd \
    openssl \
    # HTTP clients
    curl \
    wget \
    # Python
    python3 \
    python3-pip \
    python3-venv \
    # Build tools (needed by some pip packages)
    build-essential \
    git

echo "==> Adding ubuntu user to wireshark group (for non-root capture)"
sudo usermod -aG wireshark ubuntu

echo "==> Installing uv (fast Python package manager)"
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
```

```

echo "==> Creating Python virtual environment"
uv venv ~/.venv/ec441

echo "==> Installing Python packages"
source ~/.venv/ec441/bin/activate
uv pip install pyshark scapy matplotlib marimo

echo "==> Activating venv on login"
echo 'source ~/.venv/ec441/bin/activate' >> ~/.bashrc

echo "==> Installing Mininet"
sudo apt-get install -y mininet

echo "==> Done. Log out and back in for group membership (wireshark) to take effect."
echo "    Run: exit, then: multipass shell ec441"

```

What each section installs:

Package(s)	Purpose
iproute2, iputils-ping, traceroute, dnsutils	ip, ss, ping, traceroute, dig
tcpdump, tshark	CLI packet capture
iperf3	Throughput measurement
nmap	Network discovery and port scanning
netcat-openbsd	Raw TCP/UDP connections (nc)
openssl	TLS inspection (openssl s_client)
python3, python3-pip, python3-venv	Python 3 and venv support
uv	Fast Python package manager (installs pip packages)
~/.venv/ec441	Shared virtual environment; auto-activates on login
pyshark	Programmatic pcap / live capture analysis
scapy	Packet crafting and raw socket operations
matplotlib	Plotting (used in analysis notebooks)
marimo	Interactive notebooks (optional)
mininet	Network topology emulation

Step 5 — Verify the Install

After logging back in (`multipass shell ec441`), run each of these to confirm everything is working:

```

# CLI tools
ping -c 1 8.8.8.8
traceroute -m 5 8.8.8.8
dig +short google.com
ss -t
ip route

# Capture tools
tshark --version
tcpdump --version

# Throughput
iperf3 --version

# Python packages
python -c "import pyshark; print('pyshark ok')"
python -c "from scapy.all import IP; print('scapy ok')"

# Mininet
sudo mn --test pingall

```

The last command (`sudo mn --test pingall`) creates a minimal two-host Mininet topology, runs a ping between the hosts, and tears it down. You should see output ending in **Results: 0% dropped**.

Daily Use

Task	Command (run on your laptop, not inside VM)
Start the VM	<code>multipass start ec441</code>
Open a shell	<code>multipass shell ec441</code>
Stop the VM	<code>multipass stop ec441</code>
Check VM status	<code>multipass list</code>
Delete the VM	<code>multipass delete ec441 && multipass purge</code>

The VM persists between reboots — you do not need to reprovision it. Just `multipass start ec441` and `multipass shell ec441`.

Sharing files between your laptop and the VM

Multipass automatically mounts your home directory into the VM:

```

# On your laptop, files in ~ are visible at /Users/<you> inside the VM
# Or transfer explicitly:
multipass transfer myfile.pcap ec441:/home/ubuntu/

```

Troubleshooting

multipass launch fails on Windows Home Ensure virtualization is enabled in your BIOS/UEFI settings. Look for “Intel VT-x” or “AMD-V” and enable it, then restart.

Mininet prints “No default OpenFlow controller found for default switch!” This is normal. Mininet falls back to OVS bridge mode, which works correctly for all course exercises. The **Results: 0% dropped** line confirms everything is functioning.

sudo mn --test pingall fails with “RTNETLINK answers: Operation not permitted” Mininet requires full kernel namespace access. Make sure you are inside the Multipass VM (not in WSL2 or a Docker container).

import pyshark fails pyshark requires tshark to be installed at the system level. Verify:

```
which tshark
```

If missing, re-run the apt install step for **tshark**.

Scapy warning: “No libpcap provider available” Install libpcap: `sudo apt-get install -y libpcap-dev`

Notes for Instructor

- Run `sudo mn` commands inside the VM; Mininet requires root.
- Scapy raw socket operations (`send()`, `sniff()`) also require root (`sudo python3`).
- pyshark can run as a normal user for pcap file analysis; live capture requires the **wireshark** group membership set in Step 4 (takes effect after re-login).
- The **marimo** notebook server can be started inside the VM and accessed from your laptop browser via port forwarding:

```
# In the VM:
marimo edit --host 0.0.0.0 --port 2718 notebook.py
# On your laptop:
multipass info ec441    # get the VM's IP address
# Then open http://<vm-ip>:2718 in your browser
```